

FASE 1: COMPRENDE

1.- Visualización de Lista Doblemente Enlazada

- Documento con los 3 diagramas anotados por ti (puedes hacerlos en papel, ASCII o herramienta digital). Incluye una explicación con tus propias palabras de por qué eliminar un nodo conocido es $O(1)$ en la lista doble pero $O(n)$ en la lista simple.

Diagrama 1 insertar el valor 3 al inicio:

Antes de la modificación:

```

null ◀- [7] ◀-▶ [15] ◀-▶ [23] ◀-▶ [42] -▶ null

```

nodos que se modifican:

```

[3].siguiente  ─────────────────▶ [7]
[7].anterior  ◀──────────────── [3]
cabeza: [7]   ─────────────────▶ [3]

```

Después de la modificación:

Diagram illustrating a linked list structure with nodes containing values [3], [7], [15], [23], and [42]. The list starts at 'null' and ends at 'null'. The first node [3] is labeled 'cabeza' (head) and the last node [42] is labeled 'cola' (tail).

Diagrama 2 insertar el valor 50 al final:

Antes de la modificación:

```

null ← [3] ↔ [7] ↔ [15] ↔ [23] ↔ [42] → null

```

Nodos que se modifican:

```

[42].siguiente —————▶ [50]
[50].anterior ◀————— [42]
cola: [42] —————▶ [50]

```

Después de la modificación:

```

null ◀ [3] ◀→ [7] ◀→ [15] ◀→ [23] ◀→ [42] ◀→ [50] → null

```

Diagrama 3 Eliminar el nodo con valor 15 (asumiendo que tenemos el valor 15):

Antes de la modificación:

null ← [3] ↔ [7] ↔ [15] ↔ [23] ↔ [42] ↔ [50] → null

Nodos que se modifican:

[7].siguiente → [23]
[23].anterior ← [7]

Después de la modificación:

null ← [3] ↔ [7] ↔ [23] ↔ [42] ↔ [50] → null

Explicación de por qué eliminar en lista doble es $O(1)$ y en lista simple es $O(n)$:

cuando quieres eliminar un nodo en una lista doble enlazada solo tienes que reasignar dos punteros, el anterior y el siguiente del nodo que quieres eliminar, esto no depende del tamaño de la lista, sin embargo esto es distinto en las listas enlazadas simples por el hecho de que no conoces cuál es el puntero que está atrás del nodo que quieres eliminar, por lo que tienes que reasignar cada puntero de cada valor que está en la lista para tener todo en orden, así que en pocas palabras esto es así porque no te puedes desplazar hacia atrás como en las listas dobles

Checkpoint Metacognitivo - Fase COMPRENDE

- ¿Puedo dibujar de memoria la estructura de una lista doble con 3 nodos, incluyendo cabeza, cola, y todos los punteros?

definitivamente sí, es básicamente lo mismo que una lista simple solo que con flechas apuntando hacia los dos lados

- ¿Puedo explicar con mis palabras por qué `eliminarFinal()` es $O(1)$ en lista doble pero $O(n)$ en lista simple?

también puedo hacer eso, aunque no siempre es $O(n)$ cuando eliminas en una lista simple, mientras conozcas cuál es el puntero previo (por ejemplo si es cabeza) sería igual $O(1)$

- ¿Entiendo los 5 invariantes listados arriba? ¿Puedo dar un ejemplo de qué pasa si alguno se rompe?

si las entiendo ya que estas estan implicitas a la hora de trabajar con listas enlazadas dobles, si alguna de estas se rompieran no funcionaria o no seria una lista enlazada doble

FASE 2: APLICACION

2.- Proyecto: Playlist Bidireccional con Lista Doble propia:

Entregable único: Código fuente del sistema Playlist Bidireccional con la lista doble construida por ti. Incluye: (1) la clase NodoDoble y la estructura ListaDoble con mínimo 10 métodos funcionales, (2) la clase Playlist con los 5 métodos del escenario, (3) una demostración que ejecute una secuencia de operaciones mostrando inserción, navegación adelante/atrás, eliminación de canción actual y recorrido completo, (4) comentarios explicando la lógica de cada bloque y cómo el puntero "actual" se mantiene consistente.

```
#include <iostream>
#include <cstdint> // std::size_t
using namespace std;

struct Nododoble{
    string cancion;
    Nododoble * siguiente;
    Nododoble * anterior;

    explicit Nododoble( string c)
        : cancion(c), siguiente(nullptr), anterior(nullptr){}
};

class Listadoble{
public:
    Nododoble * cabeza;
    Nododoble * cola;
```

```

size_t size;

Listadoble()
: cabeza(nullptr), cola(nullptr), size(0){}

void insertarinicio(string c){
    Nododoble * nuevo = new Nododoble(c);
    if(size == 0){
        cabeza = cola = nuevo;
    } else{
        nuevo -> siguiente= cabeza;
        cabeza -> anterior= nuevo;
        cabeza =nuevo;
    }
    size++;
}

void insertarfinal(string c) {
    Nododoble * nuevo = new Nododoble(c);
    if (size == 0){
        cabeza = cola = nuevo;
    } else{
        nuevo -> anterior = cola;
        cola -> siguiente = nuevo;
        cola = nuevo;
    }
    size ++;
}

// ===== Método 3: eliminar inicio / final =====
bool eliminarinicio(){
    if (size == 0 ) return false;
    Nododoble * borrar = cabeza;

    if (size == 1 ) {
        cabeza = cola = nullptr;
    } else{
        cabeza = cabeza -> siguiente;
        cabeza -> anterior = nullptr;
    }
    delete borrar;
    size --;
    return true;
}

```

```

}

bool eliminarfinal (){
    if (size == 0) return false;

    Nododoble * borrar = cola;

    if (size == 1){
        cabeza = cola = nullptr;
    } else {
        cola = cola -> anterior;
        cola -> siguiente = nullptr;
    }
    delete borrar;
    size--;
    return true;
}

// ===== Método 4: eliminar Nodo (NodoDoble* nodo) =====

// (solo para probar) imprime de cabeza a cola
void imprimir() const {
    Nododoble* actual = cabeza;
    while (actual != nullptr) {
        cout << actual -> cancion;
        if (actual->siguiente) cout << " <-> ";
        actual = actual->siguiente;
    }
    cout << "\n";
}

// devuelve el primer nodo cuyo dato == d, o nullptr si no existe
Nododoble * buscar(string c) const{
    Nododoble * actual = cabeza;
    while (actual != nullptr){
        if (actual->cancion == c) return actual;
        actual = actual -> siguiente;
    }
    return nullptr;
}

// elimina exactamente el nodo que te pasan por puntero

```

```

bool eliminarnodo (Nododoble * nodo){
    if (nodo == nullptr || size == 0) return false;

    if (nodo == cabeza) return eliminarinicio();
    if (nodo == cola) return eliminarfinal();

    //caso "en medio": tiene anterior y siguiente
    Nododoble * a = nodo -> anterior;
    Nododoble * b = nodo -> siguiente;

    //reconectar vecinos saltandose a nodo
    a -> siguiente = b;
    b -> anterior = a;

    delete nodo;
    size--;
    return true;
}

// ===== Método 5: imprimirAtras() (recorrer de cola a cabeza) =====
void imprimiratras() const {
    Nododoble * actual = cola;
    while (actual != nullptr) {
        cout<< actual -> cancion;
        if (actual -> anterior ) cout<< " <-> ";
        actual = actual -> anterior;
    }
    cout<< "\n";
}

// ===== Método 6: vaciar() y destructor ~ListaDoble() =====
void vaciar(){
    while (size > 0){
        eliminarinicio();
    }
}

~Listadoble() {
    vaciar();
}

};

```

```
// ===== Playlist (usa la lista por dentro + puntero actual) =====

class playlist{
public:
    Listadoble canciones;
    Nododoble * actual; //cancion actual que se esta reproduciendo

    playlist() : actual (nullptr) {}

    // ETAPA 2 - 1) agregarCancion(nombre): agrega al final

    void agregarcancion(const string& nombre) {
        canciones.insertarfinal(nombre);

        //caso frontera: si era la primera cancion, "actual", debe de
        apuntar a ella
        if (actual == nullptr)
            actual = canciones.cabeza;
    }

    // ETAPA 2 - 2) siguiente(): avanza una canción

    void siguiente() {
        if (actual == nullptr) {

            cout << "playlist vacia \n";
            return; // salir para no acceder a actual->siguiente
        }

        if (actual -> siguiente == nullptr) {
            cout << "fin de la playlist \n";
            return;
        }

        actual = actual -> siguiente;
        cout << "reproduciendo: " << actual -> cancion << "\n";
    }

    // ETAPA 2 - 3) anterior(): retrocede una canción
    void anteriorcancion() {
        if (actual == nullptr) {
            cout << "playlist vacia \n"; // salir para no acceder a
            actual->anterior
            return;
        }
    }
};
```

```

    }
    if(actual -> anterior == nullptr){
        cout<< "inicio de la playlist\n";
        return;
    }
    actual = actual -> anterior;
    cout<<"reproduciendo: " << actual -> cancion << "\n";
}

// ETAPA 2 - 4) eliminarActual(): elimina la canción actual y avanza a
la siguiente

bool eliminaractual(){
    if (actual == nullptr){
        cout<< "playlist vacia\n";
        return false;
    }
    Nododoble * borrar = actual;
    //decidir a donde se movera "actual" despues de borrar
    Nododoble * nuevoactual = nullptr;
    if (actual -> siguiente != nullptr){
        nuevoactual = actual -> siguiente; //avanzar
    } else if (actual -> anterior != nullptr){
        nuevoactual = actual -> anterior; //si no se puede avanzar,
        retrocede
    } // si no hay ni siguiente ni anterior, era el unico nodo => queda
    nullptr

    bool ok = canciones.eliminarnodo(borrar);
    actual = nuevoactual;

    if (ok && actual !=nullptr){
        cout <<"reproduciendo: " <<actual -> cancion << "\n";
    } else if (ok && actual == nullptr) {
        cout<< "playlist vacia?\n";
    }

    return ok;
}

// ETAPA 2 - 5) mostrarPlaylist() - muestra toda la playlist indicando
cuál es la actual
void mostrarplaylist() const{

```



```

if (canciones.size == 0){
    cout<<"playlist vacia\n";
    return;
}

Nododoble * cur = canciones.cabeza;
while (cur != nullptr){
    if (cur == actual ){
        cout << ">> ";

    } else cout << "    ";

    cout << cur -> cancion << "\n";
    cur = cur -> siguiente;
}

};

int main(){
    playlist p;
    p.agregarcancion("intro");
    p.agregarcancion("cancion A");
    p.agregarcancion("cancion B");

    cout<< "playlist:\n";
    p.mostrarplaylist();

    cout << "\nBorro actual (Intro):\n";
    p.eliminaractual();      // debe borrar Intro y saltar a A
    p.mostrarplaylist();

    cout << "\nAvanzo y borro (B):\n";
    p.siguientecancion();    // actual: B
    p.eliminaractual();      // borra B y como no hay siguiente, salta
a A
    p.mostrarplaylist();

    cout << "\nBorro ultima que queda (A):\n";
    p.eliminaractual();      // borra A y actual queda nullptr
    p.mostrarplaylist();

```

```
return 0;  
}
```

Checkpoint Metacognitivo - Fase APLICA

¿Puedo implementar eliminarNodo() sin mirar el pseudocódigo? ¿Qué parte me cuesta más?

no se si pueda pero igual en teoria la estructura no es tan complicada de todas formas, solo es cuestion de que la estudie mas para que me la aprenda

Cuántos punteros debo actualizar al insertar en una lista doble vs. una simple? ¿Puedo listarlos?

depende, pero pues en la lista doble se ponen mas que en la simple, lo cual esto puede ser beneficioso en algunos casos

Si la IA me dio código, ¿lo entiendo línea por línea o hay partes que "confío" sin verificar?

siempre trato de entender lo que me da la ia, y si no entiendo le pido que me explique detalladamente, la cosa es estudiar todas las explicaciones que me da

FASE 3: REFLEXIONA

Reflexión Guiada

Para el sistema de playlist que implementaste: ¿Qué pasaría si hubieras usado una lista simple? ¿Cuáles operaciones serían más lentas y por cuánto? ¿El usuario notaría la diferencia con 50 canciones? ¿Y con 10,000?

pues lo mas notorio es que perderiamos el hecho de poder movernos hacia atras con la complejidad $O(1)$, y la mayoría de operaciones pasarian a ser $O(n)$ y eso no escalaria muy bien con un incremento de datos

Sobre memoria: Si cada canción ocupa 200 bytes de metadatos y un puntero ocupa 8 bytes, ¿qué porcentaje de memoria extra usa la lista doble vs. la simple para una playlist de 1,000 canciones? ¿Es significativo?

yo diria que ocuparia el doble de bytes que usa un puntero pero tal ves respecto a las canciones seria lo mismo, pero esto nos permite tener mas metodos con los que podemos reducir el tiempo de ejecucion de un programa respecto a la lista simple

Sobre tu proceso: ¿Cuál fue el bug más difícil que encontraste al implementar la lista doble? ¿Tenía que ver con olvidar actualizar un puntero? ¿Cómo lo detectaste?

no se si considerarlo un bug pero algo que se me paso y que pudo haber hecho crashear mi programa er poner un return en la parte de adelantar o retroceder canciones, ya que si no hacia esto la funcion seguia y generaba un crasheo

Checkpoint Metacognitivo - Fase REFLEXIONA

¿Puedo explicar a un compañero cuándo elegir lista doble vs. simple vs. arreglo, con ejemplos concretos?

definitivamente si, pero para resumir lista simple cosas simples lista doble cosas mas complejas

¿Qué patrón descubrí al implementar? ¿Hay una "receta" para las operaciones de lista doble?

no se muy bien a que se refiera con "receta" pero si hay algunas cosas que se repiten y pues bueno si, habia un punto donde parecia seguir un instructivo mas bien

Si tuviera que enseñar "listas dobles" a alguien, ¿por dónde empezaría y qué diagrama dibujaría primero?

empezaria por lo basico que es explicando conceptos ya despues de esos los plasmaria en un diagrama conectando cabeza con otro nodo y ese otro nodo con cola (usaria un ejemplo de tres nodos)

FASE 4: VALIDA

3.- El Auditor de Listas Dobles

Documento con las 3 implementaciones erróneas, tu análisis de cada error (dónde está, por qué es incorrecto, qué consecuencia tendría en ejecución), y la versión corregida de cada método.

Implementacion 1:

```
1  template <typename T>
2  void DoublyList<T>::insertarFinal(const T& v) {
3      Node<T>* nuevo = new Node<T>(v);
4
5      // Caso vacío: head y tail deben quedar apuntando al mismo
      nodo
6      if (head == nullptr) {
7          head = tail = nuevo;
8          ++sz;
9          return;
10     }
11
12     // Conexión estándar al final:
13     // 1) el nuevo apunta hacia atrás
14     // 2) la cola actual apunta hacia adelante al nuevo
15     // 3) actualizamos tail
16     nuevo->prev = head;    // así el nuevo "sabe" quién queda
      antes (útil para recorrer hacia atrás)
17     tail->next = nuevo;
18     tail = nuevo;
19
20     ++sz;
21 }
```

error:

nuevo->prev = head;

Implementacion 2:

```
template <typename T>
bool DoublyList<T>::eliminarNodo(const T& v) {
    if (head == nullptr) return false;

    Node<T>* cur = head;
    while (cur != nullptr && !(cur->value == v)) {
        cur = cur->next;
    }
    if (cur == nullptr) return false;
```

```

// Reengancha vecinos para "saltar" cur
if (cur->prev != nullptr) {
    cur->prev->next = cur->next;
} else {
    // Era el primero
    head = cur->next;
}

if (cur->next != nullptr) {
    cur->next->prev = cur->prev;
} else {
    // Era el último
    tail = cur->prev;
}

delete cur;
--SZ;
return true;
}

```

error: falta poner head->prev = nullptr

Implementacion 3:

```

1  template <typename T>
2  void DoublyList<T>::insertarDespuesDe(Node<T>* n, const T& v) {
3      if (n == nullptr) return;
4
5      Node<T>* nuevo = new Node<T>(v);
6
7      // Enganche hacia adelante
8      nuevo->next = n->next;
9      n->next = nuevo;
10
11     // Enganche hacia atrás (para mantener doble enlace)
12     nuevo->prev = n->prev;
13
14     // Si había alguien después, debe apuntar hacia atrás al nuevo
15     if (nuevo->next != nullptr) {
16         nuevo->next->prev = nuevo;
17     } else {
18         // Si no había siguiente, el nuevo ahora es tail
19         tail = nuevo;
20     }
21
22     ++SZ;
23 }

```

error: nuevo->prev = n->prev;

Checkpoint Metacognitivo - Fase VALIDA

¿Implementé la función verificarIntegridad() y la ejecuté después de cada operación? ¿Encontró algún error?

no la implemente asi que no

¿Pude detectar los errores en el código de la IA sin ayuda? ¿Qué estrategia usé?

nada mas en la segunda me equivoque, pero en todas me apoye en el codigo antes hecho para poder resolverlas

¿Escribí pruebas yo mismo o solo usé las que me dieron? ¿Puedo pensar en un caso de prueba que no está en la lista?

solo use los ejemplos que me dieron y de momento no puedo pensar en algun caso de prueba